



Exercises session 4

P and NP

1 Definitions

Definition 1.1 (Verifier) A **verifier** for a language L is a deterministic Turing machine V such that

$$L = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}$$

↖ certificate / witness

A **polynomial time verifier** V runs in time that is polynomial in w . A language is **polynomially verifiable** if it has a polynomial time verifier.

Definition 1.2 (NTIME)

$$NTIME(t(n)) = \{L \mid L \text{ is a language decided by a } O(t(n)) \text{ time non-deterministic TM}\}$$

Definition 1.3 (Class NP) Let's see definitions of the class NP.

1. NP is the class of all languages that have polynomial time verifiers.
2. Defining NP via NTMs

$$NP = \bigcup_k NTIME(n^k)$$

Theorem 1 A language is in NP (has a poly-time verifier) if and only if it is decided by some nondeterministic Turing machine running in polynomial time.

Definition 1.4 (Kleene star operation) Let A be a language. The operation **star** $(*)$ is defined as follows:

$$A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A \text{ where } 1 \leq i \leq k\}$$

2 Session exercises

1. Prove that the class **NP** is closed under union, intersection, kleene star and concatenation.
2. Let's G be an undirected graph. A path is considered simple if it doesn't contain repeated nodes.

- Prove that

$\text{SPATH} = \{\langle G, a, b, k \rangle : \text{there's a simple path from } a \text{ to } b \text{ of length at most } k\}$

is in **P**

- Is the following language also in **P**?

$\text{LPATH} = \{\langle G, a, b, k \rangle : \text{there's a simple path from } a \text{ to } b \text{ of length at least } k\}$

- Prove that **LPATH** is in **NP**

3. Let's introduce the SUBSET-SUM problem: given a set of integers $\mathcal{A} = \{a_1, \dots, a_n\}$ and another integer K represented in binary. Is there a subset of $\mathcal{A}$ that sums exactly K ?

- Is this language in **P**?
- Now let's think of the same problem, but the integers are represented in unary. How does our complexity analysis change?
- Prove that this problem is in **NP**

4. Given a graph $G = (V, E)$, an **independent set** of G is a set $X \subseteq V$ such that for all $(u, v) \in X \times X$, $(u, v) \notin E$.

- Give an example of a graph and two non-empty sets of vertices, such that one is an independent set and the other is not
- Prove that the following problem is in **NP**

$L = \{\langle G, k \rangle \mid G \text{ has an independent set of size } k\}$

1. Prove that the class NP is closed under union, and intersection, kleene star and concatenation.

KLEENE-STAR

- non-deterministically divide w into $x_1 x_2 \dots x_k$
- for each x_i , check if $x_i \in L$ using the TM for L .
- If all verifications accept, accept. otherwise, reject.

UNION

- L_1 and L_2 in NP, so there are certificates for both L_1 and L_2 .
Let V_1 and V_2 be such verifiers, then we can build a verifier $V_3 = V_1 \vee V_2$, and this runs in time $\max\{time(V_1), time(V_2)\}$. As both L_1 and L_2 are in NP, this is polynomial.

INTERSECTION

- same as union, but both verifiers need to accept.

CONCATENATION

- similar to the kleene-star one

4. Let's G be an undirected graph.

- Prove that

$\text{SPATH} = \{\langle G, a, b, k \rangle : \text{there's a simple path from } a \text{ to } b \text{ of length at most } k\}$

is in $\mathbf{P}$

- Is the following language also in $\mathbf{P}$?

$\text{LPATH} = \{\langle G, a, b, k \rangle : \text{there's a simple path from } a \text{ to } b \text{ of length at least } k\}$

①

Let's build an algorithm for SPATH .

We're given $\langle G, a, b, k \rangle$ and say $G = (V, E)$, $|V| = m$
The algorithm goes as follows

- ① mark node a with a 0 (zero)
- ② For $i \in [0, k]$:
- ③ for each edge (u, v) , if u is marked with i and v is unmarked, then mark v with $i+1$.
- ④ if b is marked with value $\leq k$, accept,
otherwise reject.

this is basically doing a BFS

- For goes to k ($k \leq m$)
- each edge is checked one time.

$$\begin{aligned} & \left\{ \begin{aligned} & \mathcal{O}(|V| + |E|) \\ & \text{or } \mathcal{O}(m^2) \end{aligned} \right. \end{aligned}$$

② $\text{LPATH} \in \mathbf{NP}$

- Long path

$\text{LPATH} = \{ \langle G, a, b, k \rangle : \text{there's a simple path from } a \text{ to } b \text{ of length at least } k \}$

Via NTMs

Non-deterministically choose a path of length at least k and at most $|V|$ that goes from a to b . We check in PTIME if it's a valid simple path

Via Verifier

Given a path $\pi_0 \dots \pi_l$

- check $l \geq k$
- check $\pi_0 = a$ and $\pi_l = b$
- check every π_i is unique
- check $(\pi_i, \pi_{i+1}) \in E$.

Let's introduce the SUBSET-SUM problem: given a set of integers $\mathcal{A} = \{a_1, \dots, a_n\}$ and another integer K represented in binary. Is there a subset of $\mathcal{A}$ that sums exactly K ?

- Is this language in **P**?
- Now let's think of the same problem, but the integers are represented in unary. How does our complexity analysis change?
- Prove that this problem is in **NP**

Let's build an algorithm for subset sum:
say that $|A| = m$.

How many subsets are in A ? $2^m \rightarrow$ already exponential.

But... exponential in terms of what?

↳ in the amount of numbers I have,
but... how do I encode such numbers?

Say that the input size in binary is $\underline{\underline{m}}$, then the input size in unary is $\underline{\underline{2^m}}$.

UNARY $\rightarrow$ to represent number x , we need x squares in our TM

BINARY $\rightarrow$ to represent number x , we need $\log x$ squares

Now, let's turn into SUBSET SUM again.

We can design a dynamic-programming algorithm for this problem.

(you can check it online, is not very easy to explain on paper)

BASIC IDEA:

- sort $a_1, \dots, a_n$ in ascending order
- for each a_i , we can either include it in the subset or not,
 - if included $\rightarrow \text{SUBSET}(A - \{a_i\}, K - a_i)$
 - if not, $\rightarrow \text{SUBSET}(A - \{a_i\}, K)$

if you do this in a "smart way"

it takes $\Theta(nK)$

$\hookrightarrow$ if K is in unary, then this is polynomial in terms of input

$\hookrightarrow$ if K is in binary, input is $\Theta(\log K)$
so algorithm is exponential...

	0	1	2	...	K
a_0	T	$a_1 = 1?$	$a_1 = 2?$		$a_1 = K?$
a_2	T				
$\vdots$					
a_n	T				T

MATRIX
M.

if this is
true, we
accept 😊

- the 0's column is always true, as you can form 0 with the empty set
- FOR first row, $M[0][j] = \text{true}$ if $a_0 = j$ / $j > 0$
- FOR $i \in [1, \dots, m]$
FOR $j \in [1, \dots, K]$

IF $a_i < j$:

$$M[i][j] = M[i-1][j]$$

copy
from
above

ELSE :

$$M[i][j] = \begin{cases} \text{true} & M[i-1][j] = \text{true} \\ \text{true} & M[i-1][j-a_i] = \text{true} \\ \text{false} & \sim \end{cases}$$

- Subset sum

$$\text{SUBSET-SUM} = \{ \langle A, k \rangle \mid \text{there exists } B \subseteq A \text{ such that } \sum_{b \in B} b = k \}$$

Via NTMs

non-deterministically choose a subset, and verify the sum

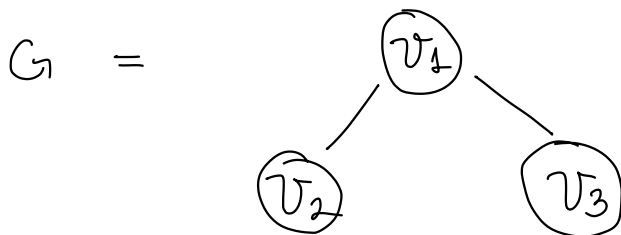
Via verifier

given a subset $B \subseteq A$, checking if it sums to k runs in $O(m)$ with $|A| = m$ and because $|B| \leq |A|$.

14) Given a graph $G = (V, E)$, an **independent set** of G is a set $X \subseteq V$ such that for all $(u, v) \in X \times X$, $(u, v) \notin E$.

- Give an example of a graph and two non-empty sets of vertices, such that one is an independent set and the other is not
- Prove that the following problem is in NP

$$L = \{ \langle G, k \rangle \mid G \text{ has an independent set of size } k \}$$



Independent sets:

$$\begin{array}{ll} \{v_1\} & \{v_2\} \\ \{v_2, v_3\} & \{v_3\} \end{array}$$

non-independent sets

$$\{v_1, v_2\} \quad \{v_1, v_3\} \quad \{v_1, v_2, v_3\}$$

To prove that is in NP

- We choose non-deterministically a set of vertices of size at least k and at most $|V|$
- We verify if its independent in $O(k^2) \in O(|V|^2)$